



Axios Network Error when making HTTP request [Solved]



Borislav Hadzhiev

Last updated: Jan 18, 2023

7 min



BobbyHadz.com

Axios Network Error when making HTTP request [Solved]



Axios Network Error when making HTTP request [Solved]

An Axios Network Error occurs for multiple reasons:

1. Your server not sending back the correct CORS headers.
2. Not specifying the protocol (``http://`` or ``https://``) when making an HTTP request.
3. Specifying an incorrect URL, port or path.

```
✖ ▶ GET http://localhost:5000/products/5 net::ERR_CONNECTION_REFUSED
✖ ▶ Uncaught (in promise)
  ▶ AxiosError {message: 'Network Error', name: 'AxiosError', code: 'ERR_
    {...}, request: XMLHttpRequest, ...}
```

```
shell
```

```
Uncaught (in promise) AxiosError {message: 'Network Error', name: 'Axio
```

Make sure to specify the protocol in the URL

Make sure you haven't forgotten to specify the protocol (``http://`` or ``https://``) in your URL when making the request.

```
shell
```

```
# ✅ correct
```

```
http://localhost:5000/posts
```

```
# ❌ incorrect
```

```
localhost:5000/posts
```

The second URL doesn't specify the protocol which causes the error.

The error is often caused by CORS issues

The most common cause of the error is CORS.

i CORS is a mechanism that allows a server to use a combination of HTTP headers to indicate from which domains, other than its own, it receives requests.

By default, servers only take requests made from applications hosted on the same domain.

If your server is hosted on `http://example.com` (or `http://localhost:5000`), for you to be able to make an HTTP request from `http://localhost:3000`, your server has to send the necessary CORS headers in its responses.

If you use Express.js for your server, you can use the CORS module to send back the correct CORS headers.

Here is an example of an Express.js API that uses the [CORS](#) module.

You can install the module with the following command.

```
shell
```

```
npm install cors
```

And use it to add CORS headers as follows.

```
index.js
```

```
const express = require('express');
const cors = require('cors');

const app = express();

// 📌 configure CORS
app.use(cors());

app.get('/products/:id', function (req, res, next) {
  res.json({msg: 'This is CORS-enabled for all origins!'});
});

const PORT = 3456;

app.listen(PORT, function () {
```

```
console.log(`CORS-enabled web server listening on port ${PORT}`);  
});
```

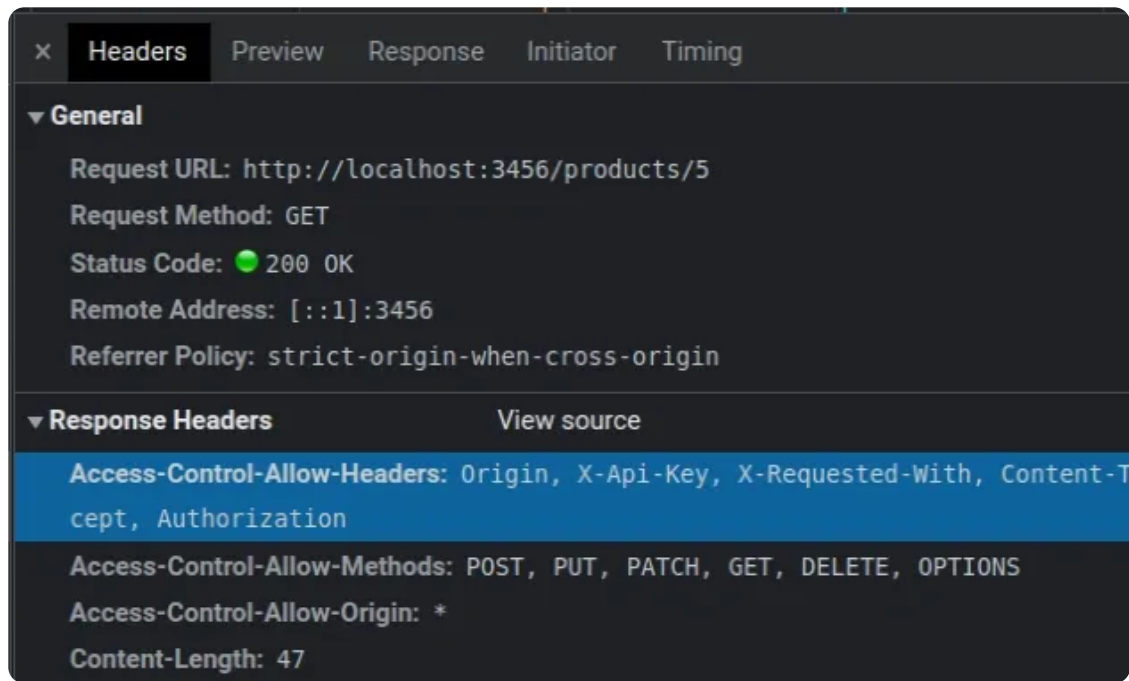
Make sure to configure CORS before sending a request to the user.

i To allow other origins to make requests to your server, you have to set the ``Access-Control-*`` headers in your server's responses.

The server should be setting the following CORS headers along with the response:

```
cors  
  
http://localhost:3000  
http://example.com  
  
POST, PUT, PATCH, GET, DELETE, OPTIONS  
  
Origin, X-API-Key, X-Requested-With, Content-Type, Accept, Authorization
```

You might have to tweak the values depending on your use case, but open the ``Network`` tab in your browser, click on the request and check if your server is setting these CORS-related headers.



The headers are:

- ``Access-Control-Allow-Origin`` - which origins are allowed to make requests to the server.
- ``Access-Control-Allow-Methods`` - which HTTP methods the origins are allowed to use when making requests to the server
- ``Access-Control-Allow-Headers`` - which HTTP headers the origins are allowed to use when making requests to the server
- ``Access-Control-Allow-Credentials`` - whether to expose the server response to the frontend when the request's credentials mode is set to ``include``. When credentials mode is set to ``include``, our frontend will always send user credentials (i.e. cookies, auth headers) even for CORS calls.

There is also an ``Access-Control-Allow-Credentials`` header. Setting it to ``true`` is only necessary if your browser sends user credentials with requests (e.g. cookies or the Authorization header).

cors

```
# 📌 only if your browser sends user credentials (cookies or Auth headers)
Access-Control-Allow-Credentials: true
```

❗ If you can't get the CORS options working, try using the `*` (asterisk) symbol as the origin and see if that works.

When an asterisk `*` is set for the `Access-Control-Allow-Origin` header, any origin on the internet can access the server.

cors

```
Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: POST, PUT, PATCH, GET, DELETE, OPTIONS

Access-Control-Allow-Headers: *
```

You would want to narrow this down in production, but it's a useful tool when debugging.

Note that the `Access-Control-Allow-Credentials` header cannot be set to `true` if `Access-Control-Allow-Origin` is set to an asterisk `*`.

When the `Access-Control-Allow-Headers` is set to an asterisk, all headers are allowed in a preflight request.

Make sure that the URL you're passing to `axios` is correct

Make sure that the URL you've specified when making the HTTP request is correct and complete.

- The URL has to include the protocol, e.g. ``http://`` or ``https://`` if you're testing on localhost without an SSL certificate.
- The path has to be correct e.g. ``/articles``.
- The HTTP method (e.g. ``GET`` or ``POST``) has to be correct for the specific path (e.g. ``/articles``).

i If you misspell any of the configurations when making the HTTP request, e.g. a property in the headers object or the HTTP method, the error occurs.

Here is an example HTTP request using ``axios`` and ``react``.

App.js

```
import axios from 'axios';
import {useEffect} from 'react';

function App() {
  useEffect(() => {
    axios.get('http://localhost:3456/products/5').then(response => {
      console.log(response.data);
    });
  }, []);

  return (
    <div className="App">
      <h2>Hello world</h2>
    </div>
  );
}

export default App;
```

If the URL, path, HTTP method, headers and body of your HTTP request are correct, the most likely cause of the error is your server not sending the correct CORS HTTP headers that would allow your browser to make an HTTP request.

Try setting the following HTTP response headers on your server.

cors

```
# 📌 your domain below, e.g. http://localhost:3000
Access-Control-Allow-Origin: http://example.com

Access-Control-Allow-Methods: POST, PUT, PATCH, GET, DELETE, OPTIONS

Access-Control-Allow-Headers: Origin, X-API-Key, X-Requested-With, Content-Type
```

If that doesn't work, set the `Access-Control-Allow-Origin` to an asterisk to allow HTTP requests from all origins and make an HTTP request to the server again.

cors

```
Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: POST, PUT, PATCH, GET, DELETE, OPTIONS

Access-Control-Allow-Headers: *
```

Once you get CORS working, you can narrow down the values of the `Access-Control-Allow-*` headers.

If you don't have access to the server, use a proxy

A proxy is a server that sits between the client (browser) and the server you need to make an HTTP request to.

If you aren't able to set the `Access-Control-Allow-*` response headers on the server, you can make an HTTP request to a proxy, which makes an HTTP request to the other server.

This is possible because the same origin policy isn't enforced when making requests from one server to another.

i If your proxy server sends the correct ``Access-Control-Allow-*`` headers to your client (browser), then it can just fetch the information from the other server and respond with it.

Here is an example of a simple proxy using Express.js.

In order to use it, you first have to install the dependencies.

```
shell
```

```
npm install express
```

The proxy consists of the following code stored in an ``index.js`` file.

```
index.js
```

```
const express = require('express');

const app = express();

app.use((_req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Headers', '*');

  next();
});

app.get('/users', async (_req, res) => {
  const url = 'https://randomuser.me/api/';

  try {
    const response = await fetch(url);

    if (!response.ok) {
```

```
        throw new Error(`Error! status: ${response.status}`);
    }

    const result = await response.json();
    return res.json(result);
} catch (error) {
    console.log(error);
    return res.status(500).json({error: 'An error occurred'});
}
});

const port = 3456;

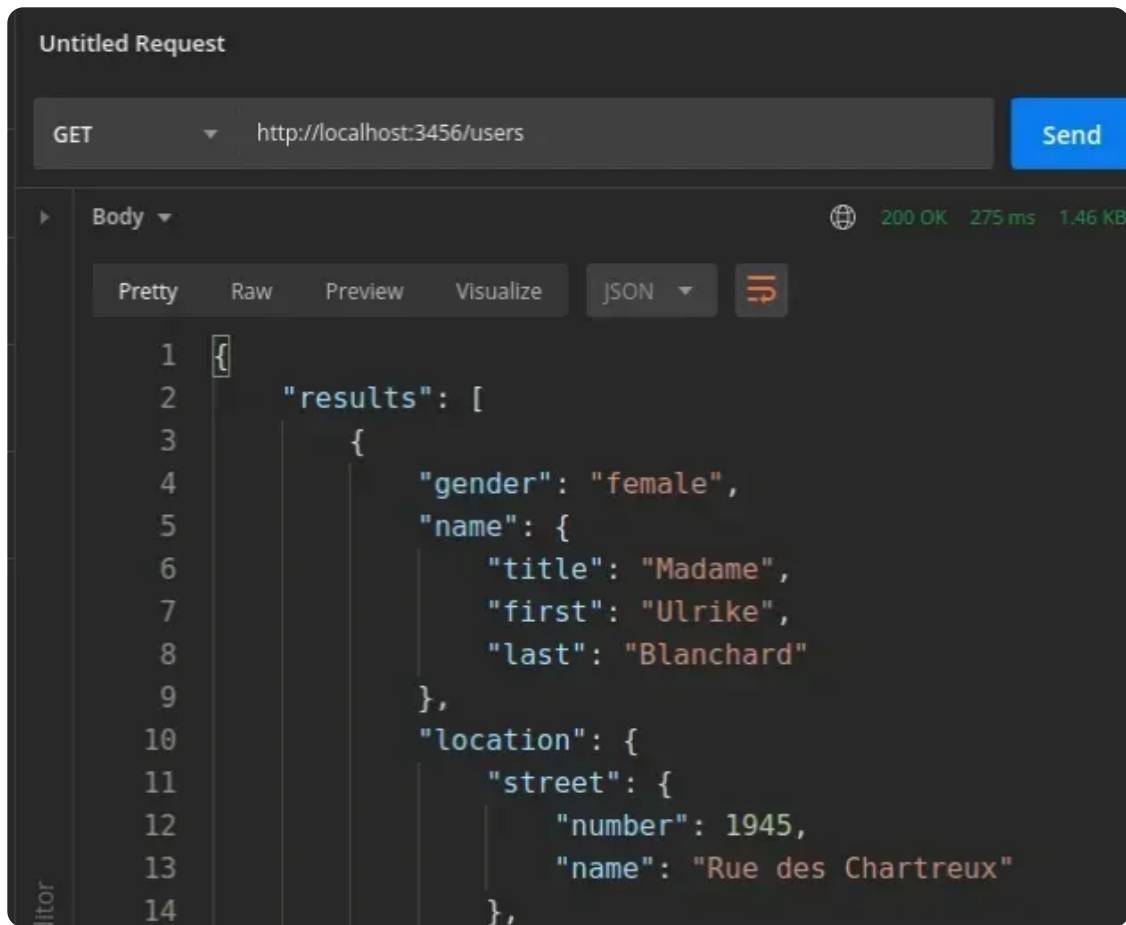
app.listen(port, () =>
    console.log(`Server running on http://localhost:${port}`),
);
```

You can start the proxy server with the `node index.js` command.

```
shell
```

```
node index.js
```

You can make an HTTP request to <http://localhost:3456/users> to get a response with the data.



We set the ``Access-Control-Allow-Origin`` and ``Access-Control-Allow-Headers`` headers to an asterisk to allow all origins to make requests to the proxy server.

The next step is to make an HTTP request to the other server and send the response to the client.

i The proxy server simply sits between the browser and the other server and takes advantage of the fact that one server can make an HTTP request to another without having to pass CORS checks.

Alternatively, you can use a proxy server that makes an HTTP request to the specified URL and responds with the result.

The [cors-anywhere](https://github.com/RobyT/cors-anywhere) package is a NodeJS proxy which adds CORS headers to the proxied request.

Using the cors-anywhere package as a proxy

You can use the following command to install the `cors-anywhere` package.

```
shell
```

```
npm install cors-anywhere
```

Here are the contents of a simple implementation of the proxy server stored in an `index.js` file.

```
index.js
```

```
const cors_proxy = require('cors-anywhere');

const host = process.env.HOST || '0.0.0.0';

// Listen on a specific port via the PORT environment variable
const port = process.env.PORT || 8080;

cors_proxy
  .createServer({
    originWhitelist: [], // Allow all origins
  })
  .listen(port, host, function () {
    console.log('Running CORS Anywhere on ' + host + ':' + port);
  });
```

You can start the proxy server with the `node index.js` command.

```
shell
```

```
node index.js
```

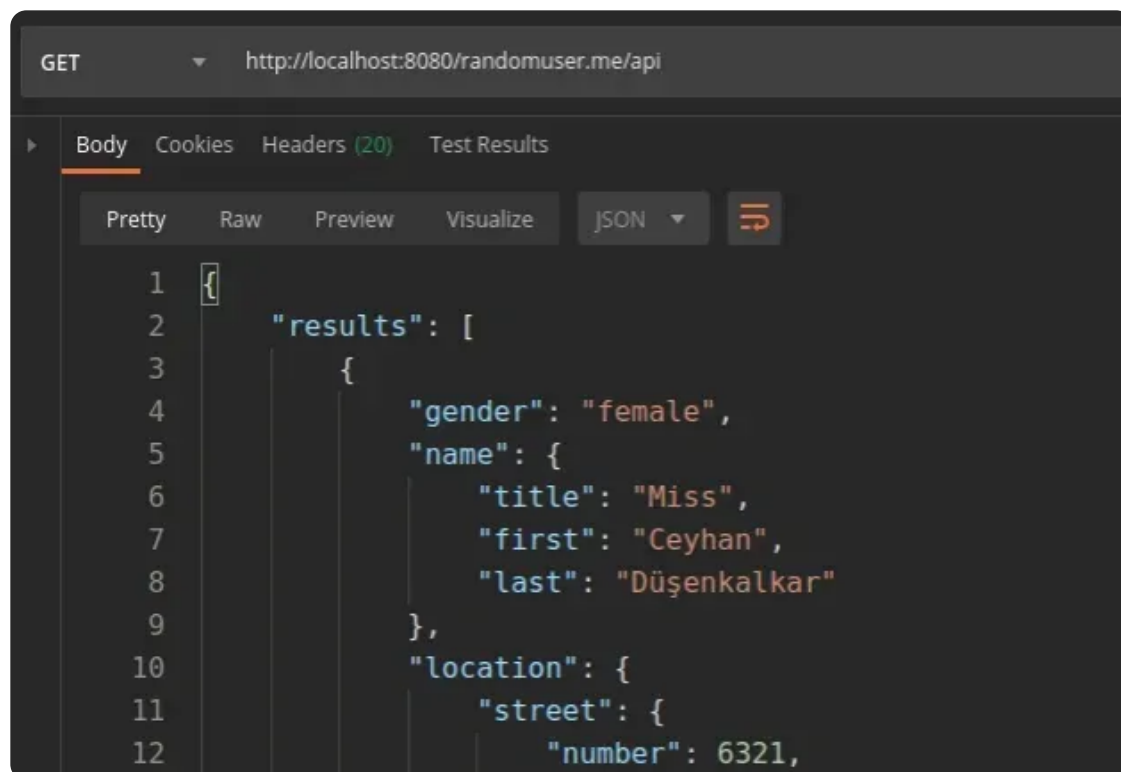
The server is available at `http://localhost:8080`.

As shown in [the docs](#), the URL to proxy is taken from the path.

Here are some examples.

```
shell
```

```
# 📌 makes a request to randomuser.me/api  
http://localhost:8080/randomuser.me/api  
  
# 📌 makes a request to google.com  
http://localhost:8080/google.com
```



The concept is the same - the proxy server makes an HTTP request to the specified URL and sends the response data back to the client.

I've also written a detailed guide on [making HTTP requests with Axios in TypeScript](#).

If you get an error that [ReferenceError: process is not defined](#), click on the link and follow the instructions.

Conclusion

To solve the "Axios Network Error", make sure:

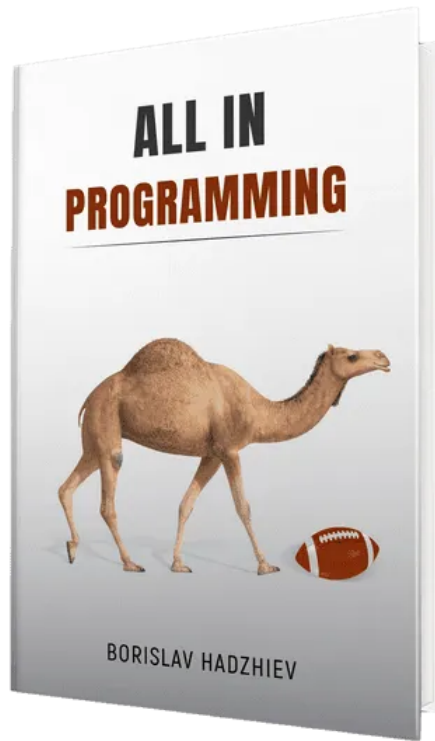
1. Your server sends back all the necessary CORS headers to enable requests from a different origin.
2. You haven't specified an incorrect or incomplete URL when making the request.
3. You haven't specified the wrong protocol in the URL.
4. You haven't specified a wrong HTTP method or headers when making the request.

Additional Resources

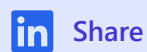
You can learn more about the related topics by checking out the following tutorials:

- [React Hook useEffect has a missing dependency error \[Fixed\]](#)
- [useEffect must not return anything besides a function](#)
- [Can't perform a react state update on an unmounted component](#)
- [TypeError: Failed to fetch and CORS in JavaScript](#)
- [Fetch API cannot load localhost. URL scheme is not supported](#)
- [Cannot update a component while rendering a different component](#)
- [How to convert an Object to FormData in JavaScript](#)
- [Failed to parse source map from X error in React \[Solved\]](#)
- [No QueryClient set, use QueryClientProvider to set one](#)
- [How to handle Timeouts when using Axios \[3 easy Ways\]](#)
- [TypeError: Failed to execute 'fetch' on 'Window' \[Solved\]](#)
- [How to use Basic Auth with Axios in JavaScript](#)

- ✓ I wrote [a book](#) in which I share everything I know about how to become a better, more efficient programmer.



- ✓ You can use the search field on my [Home Page](#) to filter through all of my articles.



[About](#)

[Contacts](#)

[Policy](#)

[Terms & Conditions](#)



Copyright © 2023 Borislav Hadzhiev